

HelixSpecifier

HelixSpecifier

Spec-driven development that scales its own ceremony to the work.

Summary

HelixSpecifier is a Go engine that fuses three development methodologies — SpecKit's spec-driven workflow, Superpowers' TDD discipline, and GSD's milestone lifecycle — into one adaptive flow. It classifies each task by effort and scales the amount of process up or down accordingly.

Short description

HelixSpecifier is a spec-driven development fusion engine for AI agents. It combines SpecKit, Superpowers and GSD, classifies work by effort level, runs debate-backed specification phases, enforces a minimum test-to-code ratio, and learns from every completed flow.

Long description

HelixSpecifier is a spec-driven development (SDD) fusion engine written in Go (module `digital.vasic.helixspecifier`), built as a component of the HelixAgent AI ensemble. It takes three development practices that normally live in three separate tools — and three separate mindsets — and fuses them into one adaptive workflow: SpecKit's seven-phase SDD process (Constitution, Specify, Clarify, Plan, Tasks, Analyze, Implement), Superpowers' test-driven discipline with parallel subagent execution, and GSD's milestone lifecycle management. Each pillar keeps doing what it is good at; the engine is what makes them run as a single coherent flow instead of a hand-stitched pipeline.

Its central idea is *adaptive ceremony*: the engine classifies incoming work by effort level and scales the amount of process to match, so a one-line fix isn't dragged through the same heavyweight ritual as a major feature — and a major feature isn't waved through with the rigor of a typo. Ten power features build on that spine: bounded-concurrency parallel task execution, a machine-readable “Constitution as Code” that enforces mandatory rules automatically, “Nyquist TDD” that tracks and enforces a minimum test-to-implementation ratio, multi-round multi-agent debate for specification refinement, adaptive skill-proficiency learning, brownfield analysis of legacy code, predictive specification drawn from historical patterns, cross-project knowledge transfer, runtime ceremony adjustment that re-tunes mid-flight, and a persistent spec memory with semantic search.

It is consumed as a Go module — via `go get` or a local `replace` directive — behind a deliberately small engine API: register the three pillars plus a ceremony scaler and spec memory, classify the effort of the work, then execute the full flow and receive a quality-scored result. The surface is simple; the orchestration behind it is not. Like the rest of the Helix family, it is developed under an anti-bluff verification regime, with an in-process challenge runner exercising real code rather than mocks.

Why we built it

Spec-driven development, rigorous TDD, and milestone management are usually three separate practices with three separate tools. HelixSpecifier was built so an AI agent (HelixAgent) can run all three as one coherent, self-scaling workflow instead of stitching them together by hand.

Why it's a game-changer

It makes process proportional to the work — automatically. Teams normally get stuck at one of two bad extremes: heavy ceremony on everything (safe but slow, and quietly resented) or ceremony on nothing (fast until it isn't). HelixSpecifier dissolves that trade-off by sizing the ceremony to the classified effort of each task, and re-tuning it at runtime as the work reveals itself. The capability that wasn't practical before is process that right-sizes itself per task — and, on top of that, specification decisions backed by a multi-round, multi-agent debate with position scoring rather than a single agent's first guess.

What's innovative

- **Adaptive ceremony** — process level driven by real-time quality metrics and adjusted at runtime, not fixed up front.
- **Nyquist TDD** — a test-to-implementation ratio gate (minimum 2x), borrowing the Nyquist sampling theorem's logic: to faithfully capture behavior you must sample it well above its rate, so tests must out-measure the code they cover.
- **Debate architecture** — multi-round, multi-agent specification refinement where positions are proposed, scored, and converged, replacing a single opinion with an adversarial one.
- **Predictive specification** and **cross-project transfer** — the engine mines accumulated flows to anticipate specs and carry hard-won knowledge from one project into the next.
- **Constitution as Code** — mandatory project rules made machine-readable and enforced by the engine, not left to reviewer vigilance.

Biggest technical challenges & how we solved them

- **Merging three methodologies without them fighting each other** — SpecKit, Superpowers, and GSD each assume they own the workflow. Solved with a fusion engine that registers each pillar behind a common interface and drives them through one shared flow lifecycle, so they compose into a single process instead of three that collide.
- **Deciding how much process a given task actually needs** — guess too high and everything crawls; too low and risky work ships unchecked. Solved with an effort classifier that sizes the work, feeding a ceremony scaler that adjusts the process level dynamically as execution proceeds.
- **Keeping specification quality high without a human gatekeeper on every decision** — solved by replacing single-shot specs with debate-backed refinement, where agents score competing positions across rounds, and by enforcing Nyquist TDD ratios so implementation can't outrun its tests.

Tech stack

- **Go** — chosen so the engine ships as a single importable binary with no runtime baggage; its concurrency model is what makes bounded-parallel task dispatch and the multi-agent debate rounds tractable rather than a threading headache.
- **logrus** — structured logging threaded across the engine and all three pillars, so a flow's decisions (classification, ceremony changes, debate outcomes) are legible after the fact.

- **SpecKit pillar** — the seven-phase spec-driven-development process (Constitution → Specify → Clarify → Plan → Tasks → Analyze → Implement), providing the disciplined backbone of how a spec becomes code.
- **Superpowers pillar** — TDD discipline with parallel subagent execution, supplying the test-first rigor and the fan-out that keeps implementation honest and fast.
- **GSD pillar** — milestone and lifecycle management, giving the flow its sense of “done” and its progression through stages.
- **Spec memory store** — a persistent, semantically searchable index of past specifications, the substrate that makes predictive specification and cross-project transfer possible instead of starting cold every time.

Status & honesty notes

- **Status: beta.** Consumed as a Go module component of HelixAgent.
- **License: TBD.** No LICENSE was detected via the GitHub API — UNVERIFIED / not declared.
- The display name “HelixSpecifier” maps to the repository specifier.

Priority tier: Helix-primary.